

Coding & Solution

Date	1 October 2026
Team ID	SWTID-2026-4934
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	5 Marks

Coding & Solution

This section summarizes the implemented solution: a High Impact Control UI Policy and three client scripts on the Incident form that keep Incident data complete and consistent before it is saved. The solution works as designed and was verified with five test cases, all of which passed.

How the Solution Meets the Requirements:

1. The UI Policy and the three client scripts are built with ServiceNow configuration and a few lines of JavaScript, and each component has a clear, descriptive name.
2. The solution addresses the core problem: Incident data is checked while it is being entered. A High impact incident needs an Assignment group and an Assigned to value, Urgency follows Impact automatically, and State cannot be changed by a quick edit in the list.
3. The scripts are listed in the Appendix of the project report and are configured on the ServiceNow instance, and the setup and run steps are given in the table below.
4. The project documents and screenshots are kept in the GitHub repository linked in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Rash6906/Implement-Client-Script-and-UI-Policy-Incident-.git
Programming Language(s)	JavaScript for the three client scripts (using the GlideForm g_form API). The UI Policy and its actions are configured declaratively, with no code.
Framework(s) Used	ServiceNow Incident Management, UI Policies and Client Scripts
Key Features Implemented	• UI Policy "High Impact Control" on the Incident table, applied when Impact is 1 - High, with Reverse if false enabled • UI Policy Action that makes Assignment group mandatory while the condition is met • UI Policy Action that makes Urgency read-only while the condition is met • onChange script "Auto set urgency for high impact" that sets Urgency to 1 - High and shows an information message • onSubmit script "Prevent save if assigned to missing" that blocks saving a High impact incident without Assigned to and shows an error on that field • onCellEdit script "Prevent state change via list edit" that shows an alert and cancels State edits made in the list, while form edits stay allowed • Five test cases (TC-01 to TC-05) run on the instance, all passed
Pending / Incomplete Features	• Reset Urgency when Impact is lowered from High • Role-based exceptions for the list-edit block on State • Show Assigned to as mandatory on the form itself, not only at submit • Server-side controls for records created by imports or integrations • Script descriptions, inline comments and configurable messages

Field	Details
Setup / Run Instructions	1. Log in to the ServiceNow instance (dev448044.service-now.com) with administrator or configuration access. 2. Open All > System UI > UI Policies and open "High Impact Control". Check that it is Active on the Incident table with the condition Impact is 1 - High and Reverse if false selected, and that its two actions (Assignment group mandatory, Urgency read-only) are listed. 3. Open All > System Definition > Client Scripts and check that the three scripts are Active: "Auto set urgency for high impact" (onChange), "Prevent save if assigned to missing" (onSubmit) and "Prevent state change via list edit" (onCellEdit). 4. Open All > Incident > Create New and set Impact to 1 - High. Urgency becomes 1 - High and read-only, and Assignment group becomes mandatory. 5. Leave Assigned to empty and click Submit. The save is blocked and an error appears under Assigned to. Select a user in Assigned to and submit again to save the incident. 6. Change Impact to 2 - Medium. Urgency becomes editable and Assigned to is no longer enforced. 7. Open the Incident list and double-click State on any record. An alert appears and the change is cancelled. Then open the incident and change State on the form, which saves normally.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	No
4	Error handling is implemented for critical operations	No
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

- The solution is built mainly by configuration in ServiceNow, with three short JavaScript client scripts. The scripts are listed in the project report, and the project documents and screenshots are kept in the GitHub repository.
- The UI Policy and each client script are separate, clearly named components, and each script has a single purpose.
- Not done yet: the scripts have no inline comments and their Description fields are empty, and there is no try/catch or server-side check. These are listed as future improvements.
- ServiceNow showed a warning that more than one UI Policy has the same Order for the Urgency field (Figure 4 of the project report). All five test cases still passed.